RULE / STATE / EFFECT / PROOF

Business Logic Vulnerabilities Explained

Find broken rules, prove the outcome, and build workflows that stay correct

SUPPORTED RULE

What must remain true?

ACTUAL EFFECT

What did the server grant?

Follow the promise from input to protected outcome.

Find the weakness. Prove the effect. Verify the repair.

Practice with real SQLite and bounded local workflows.

Haroon Shahid

First edition / Version 1.0 / 2 November 2026

Before you begin

Business Logic Vulnerabilities Explained

Find broken rules, prove the outcome, and build workflows that stay correct

By Haroon Shahid. First edition, version 1.0. 2 November 2026. Prepared with AI assistance.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under LICENSE.txt. The separately identified lab code is MIT licensed. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. RuleForge uses real SQLite transactions, constraints, HMAC event checks and fixed-loopback HTTP with fictional actors and LAB credits. Global visibility, issuer fixtures, reset, clock and scheduling controls are privileged teaching tools. Provider, framework, inventory, subscription, broker and distributed-system chapters include review guidance and synthetic cases; those deployments are not implemented by this model. It is not a production commerce or security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format. On narrow screens, swipe sideways within wide diagrams and tables. No full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package includes all 64 chapters, answered reviews, the offline reading edition, practice workbooks, reference sheets, and the local RuleForge lab.

Contents

Before you begin 2
Contents 3
Introduction 4
Learning roadmap 5
01. Follow the promise behind a feature 7
02. Read the conversation behind the screen 9
03. Separate identity, permission and workflow authority 11
Sources and further reading 13

Introduction

Mira buys one notebook from Aster Market. The server accepts her request, records a payment and creates a delivery. Everything looks successful.

One detail is wrong: Mira chose the unit price herself.

The application can parse valid data, calculate correctly and process payment consistently while breaking its own promise. That is why business logic review starts with meaning. What should this actor be allowed to obtain, under which state and evidence?

One journey connects the book

We follow quotes, orders, payment, delivery, refunds, rewards and independent approvals. Start with requests, types and state. Later, those foundations explain concurrent claims, retry identity, worker snapshots, tenant boundaries and partial failure.

The examples use fictional LAB credits. No real provider, money or shipment is required. Aster Market is the story; RuleForge is the optional local laboratory.

Four questions to remember

1. **Rule:** What must remain true, including legitimate exceptions?
2. **State:** Which actor, scope, values, version and history matter?
3. **Effect:** What does the actual consumer grant or change?
4. **Proof:** Which observation supports the conclusion, and what limits it?

Think of a ticket office. A readable ticket, a valid payment and permission for a particular seat are different facts. Repeating a receipt does not necessarily buy another seat. The same distinctions matter when software makes the decisions.

What makes the practice useful

RuleForge performs real SQLite transactions, constraints, HMAC verification, local HTTP and competing calls. Weak paths sit beside stricter paths. You can inspect an unauthorized cheap purchase, an excessive refund, repeated promotion value or stale approval effect.

Its global state, actor picker, signed negative fixtures, time controls and scheduling barrier are privileged teaching facilities. They do not establish ordinary attacker capabilities. The validation report separates executed local behavior from framework, provider and distributed-system review guidance.

You will also practice saying no to unsupported findings. A valid price lock, a cached retry result or a downstream rejection can be correct. Accurate limits belong in the explanation.

Use the book actively

Predict before revealing. Explain a diagram aloud. Try one mission. Write a fact, an inference and a limit. Return later and analyze a different case without copying its field names.

Use only systems and accounts you own or have explicit permission to assess. Keep proofs small and reversible. Finishing this self-study kit does not certify expertise or guarantee discoveries or income. Its goal is a skill you can demonstrate: follow a promise into a decision, inspect its effect and defend the proof.

Learning roadmap

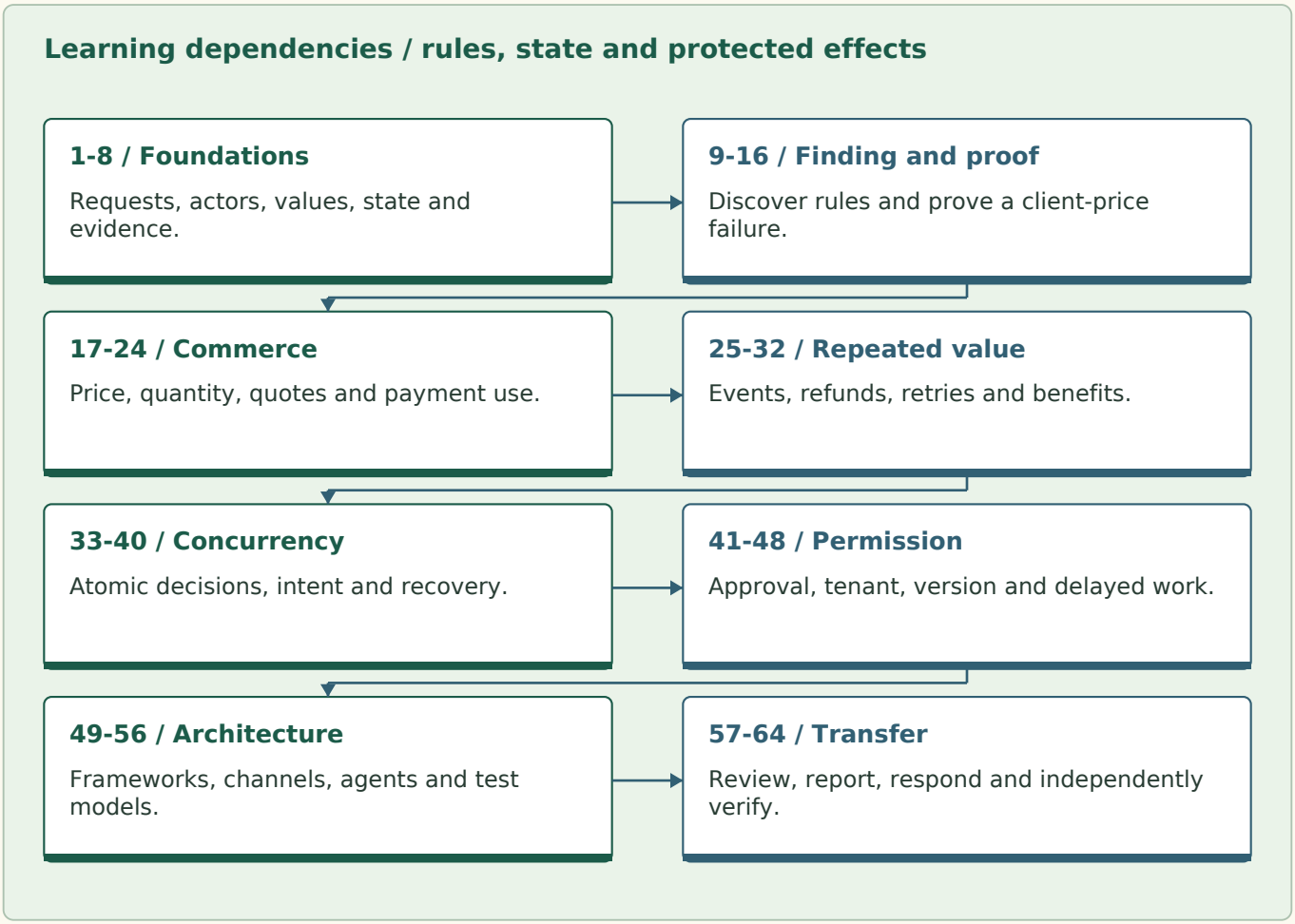
Choose a starting route

- New learner: Chapters 1-8, then the first missions. Explain requests, actors, rules and protected effects.

- Developer: foundations, then Chapters 17-40 and 49-56. Write invariant, commit and recovery contracts.

- Authorized tester: Chapters 8-16, then lifecycle sections. Produce paired tests and a clean evidence trail.

- Architect: Chapters 33-56 after the evidence model. Map rule ownership, identities and external boundaries.



Read left to right within each stage, then follow the connector to the next row. Each later stage uses the earlier mental model.

Eight stages, one connected model

Stage	Chapters	Observable milestone
Foundations	1-8	Separate valid data, identity, rule and final outcome
Finding and proof	9-16	Demonstrate an unauthorized client-price chain
Commerce	17-24	Bind prices, quantities, quote snapshots and payment
Repeated value	25-32	Enforce event, refund, retry and benefit lifecycles
Concurrent systems	33-40	Explain atomic decisions, outbox and recovery limits
Permission workflows	41-48	Preserve independent approval, scope and version
Real architecture	49-56	Review actual frameworks, channels, agents and tests
Transfer	57-64	Produce an independent defensible report and retest plan

Practice order

Start with missions 1-3 after the foundations and first-finding chapters. Continue through 4-16 for values, quotes and state. Use 17-26 for event evidence and workers, 27-34 for refunds/rewards, 35-36 for concurrency and 37-39 for approval. Mission 40 runs the suite and transfers the method to the capstone. Each mission states its own setup; return to the relevant chapter whenever a result is hard to explain.

The 56 predictions train judgment before execution. The 32 synthetic review cases test transfer to designs the local lab does not implement. Every chapter has three questions with explained answers and a rule/state/effect/proof revision diagram. The separate visual atlas collects all 64 revision views.

Retain the skill

After a day, redraw one workflow without notes. After a week, analyze a new case. After a month, write a small independent report. Use the mastery tracker to select a missing piece of evidence rather than award yourself a professional title. A supported explanation is more useful than time spent or a checklist score.

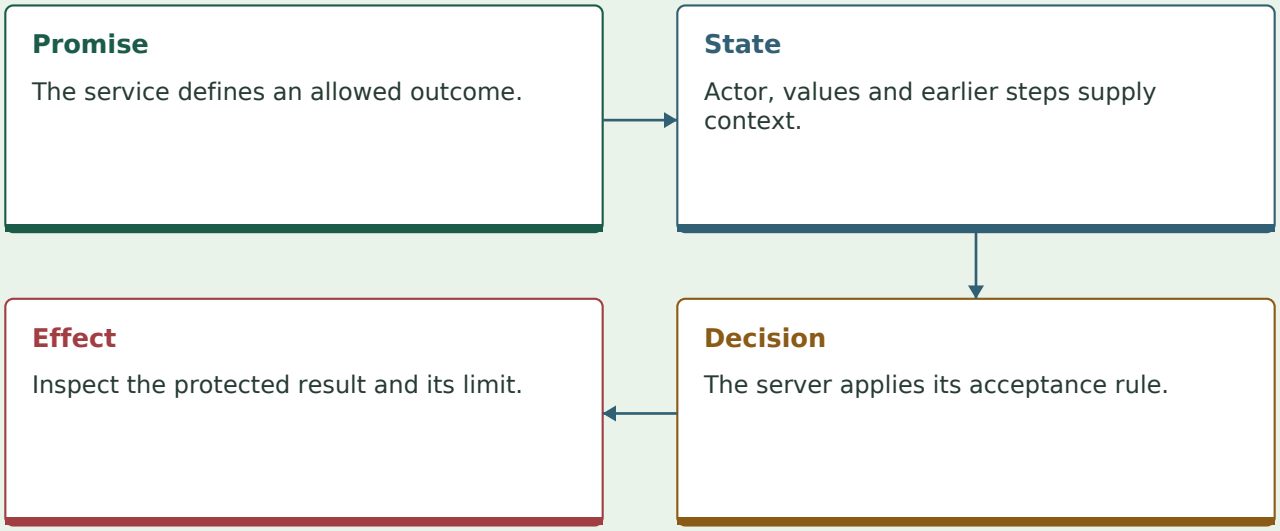
I. UNDERSTAND RULES, STATE AND EVIDENCE

01. Follow the promise behind a feature

Learn to: Define a business rule and separate unusual behavior from a demonstrated broken promise.

Think of it this way: A ticket machine can print a perfectly readable ticket for the wrong journey. The printing worked; the permission did not.

01A / A feature connects a rule to an observable effect



Name the rule before judging the behavior; prove the protected outcome before claiming impact.

A feature makes a promise

Mira buys a notebook from Aster Market, our fictional service. The catalog price is ten LAB credits. LAB is a made-up teaching currency. One credit contains one hundred minor units. No real payment is involved.

The service promises to charge the authorized price, confirm payment, and deliver the purchased item once. Each sentence describes a **business rule**: a condition that defines how this application should work. Business rules also appear in school portals, games, document approvals and free services. They need not involve a company or money.

A **business logic vulnerability** allows an actor to break a security-relevant application rule through the application’s behavior. The request may be valid JSON. The user may be correctly signed in. The resulting action can still be wrong.

Start with an outcome

Suppose Mira changes a submitted unit price from ten credits to one hundredth of a credit. The server creates the cheap order, accepts the tiny fictional payment and records the normal notebook delivery. The problem is not that an editable field existed. The problem is that the service accepted the user’s price as authoritative and provided the protected outcome.

Now suppose the store offers an approved ninety-nine-percent discount. The same low price could be correct. Context changes the conclusion. You need the rule, the actor’s eligibility, the accepted values and the actual result.

The normal journey is your first tool

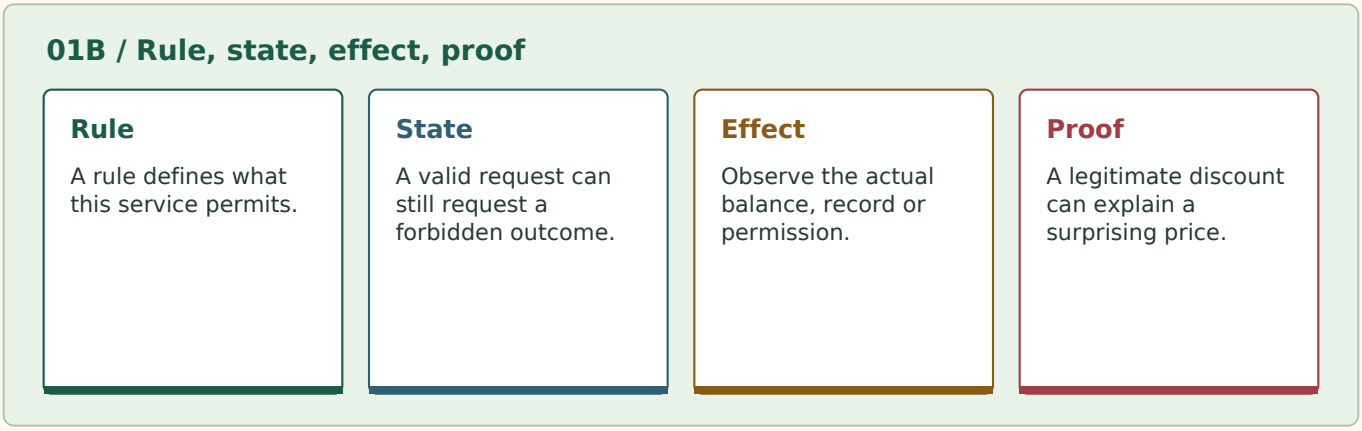
Use a feature normally before changing it. Follow the cart, quote, order, payment, delivery, refund and reward. A **workflow** is a sequence of related actions. Each action changes or reads **state**, the application’s current remembered facts.

An order can be pending, paid or cancelled. A reward may require a paid purchase. An approval may apply to a particular version. These states connect the screens to server decisions.

Four questions to keep

Ask what must remain true, what state matters, what effect occurred, and what proves your conclusion. A success message is weaker evidence than a changed balance or delivered dummy item. An unexpected response is a lead until you understand its effect.

In this book you will find weaknesses, prove them in a bounded local model, and repair the underlying decisions. You will also practice rejecting attractive but unsupported findings. Professional confidence comes from precise evidence, not from calling every surprise a vulnerability.



Key takeaway. Name the rule before judging the behavior; prove the protected outcome before claiming impact.

Check your understanding

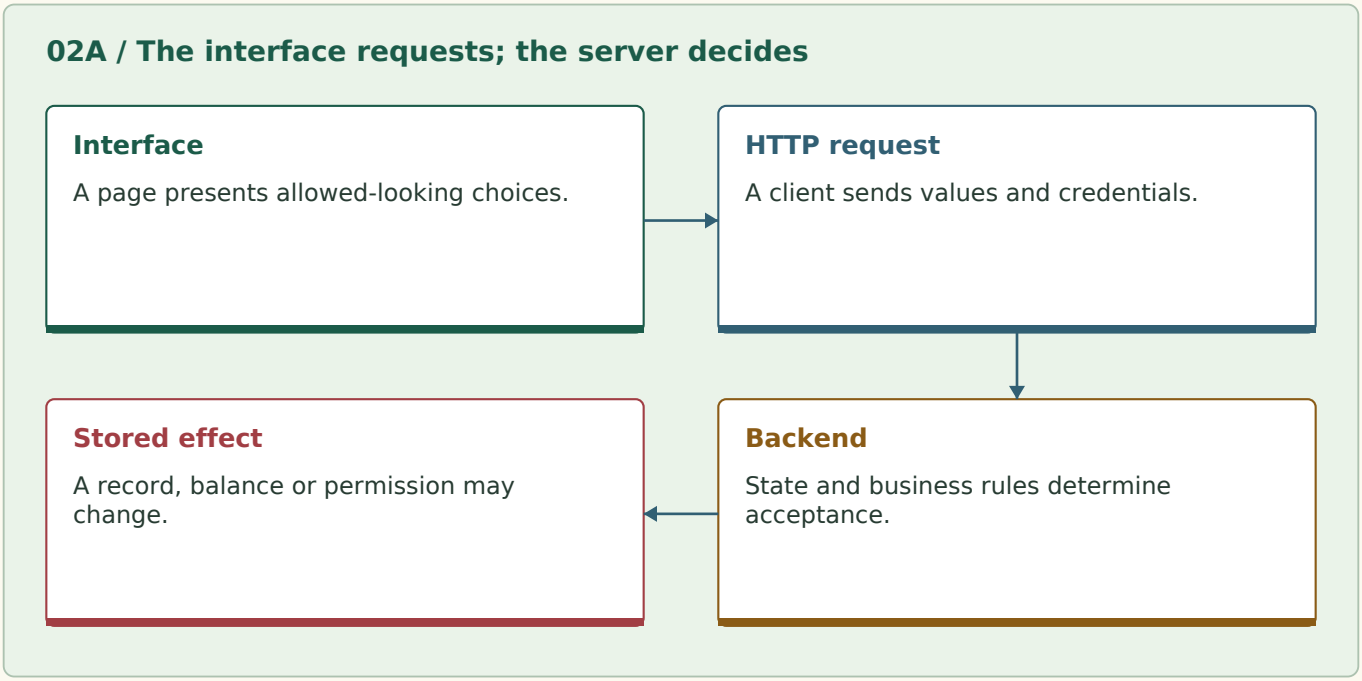
1.1 Does an editable price field prove a bug?	**1.2** Must a business logic flaw involve money?	**1.3** Why study normal use first?

Answer explanations are in the full ebook. Sources: LOGIC, WSTG, BUSINESS

02. Read the conversation behind the screen

Learn to: Understand clients, servers, APIs, HTTP requests and why a page is not the enforcement boundary.

Think of it this way: A restaurant menu shows choices, but the kitchen and billing system decide what is prepared and charged.



Follow the request into the server decision and inspect what changed beyond the screen.

Two programs exchange messages

A browser is a client: software that asks another system to do work. A server receives requests and returns responses. The backend is the application logic behind those requests. A database stores remembered records. These are roles; one computer can perform several of them.

A URL is an address. In http://127.0.0.1:8970/api/quote, http is the scheme, 127.0.0.1 is the host, 8970 is the port and /api/quote is the path. Loopback refers to your own computer. DNS normally translates readable hostnames into network addresses. The lab uses a fixed loopback address so it needs no external name lookup.

HTTP is the web's request/response protocol. A request has a method, target, headers and sometimes a body. A response has a status, headers and sometimes a body. An API is a defined interface that software uses to request operations or data.

```
POST /api/order-create HTTP/1.1
Host: 127.0.0.1:8970
Content-Type: application/json

{"quote":"fictional-quote-id"}
```

This is a message sketch. A real client supplies framing, cookies and the local lab's required control headers.

The browser is not the only possible client

A page may disable a negative quantity or hide the refund button. Another client can still submit a request directly. Therefore the server must check quantity, account, state and permission independently of the interface.

A frontend is code concerned with the client experience. React, Vue and similar tools can make helpful interfaces. Their visible controls cannot force every caller to behave like that interface. A server route must enforce the actual rule.

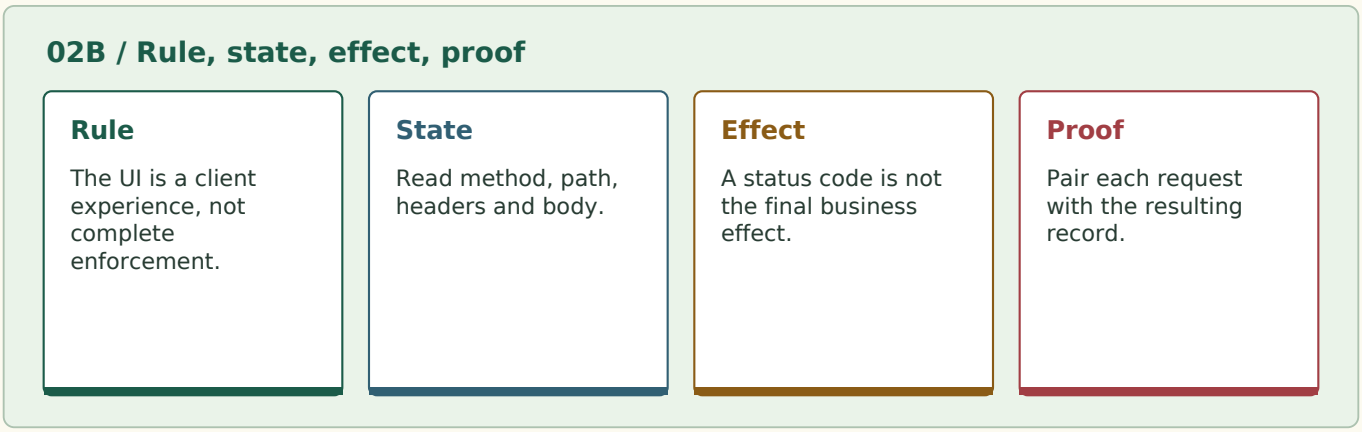
Read past the status

A response of 200 means the server chose a successful HTTP status; it does not independently prove payment, delivery or authorization. The body may contain an error or an intermediate result. A redirect may send the user to a confirmation page while the authoritative payment remains pending.

HTTPS protects transport using TLS. It does not prevent the application from trusting a false price or applying the same refund twice. Correct transport and correct application decisions solve different problems.

Practice the connection

In browser developer tools, open the Network panel and perform one normal local operation. Find its request and response. Match the operation to the changed server record. Later you can repeat authorized requests using a proxy or the lab UI. First learn which message actually causes the effect you want to examine.



Key takeaway. Follow the request into the server decision and inspect what changed beyond the screen.

Check your understanding

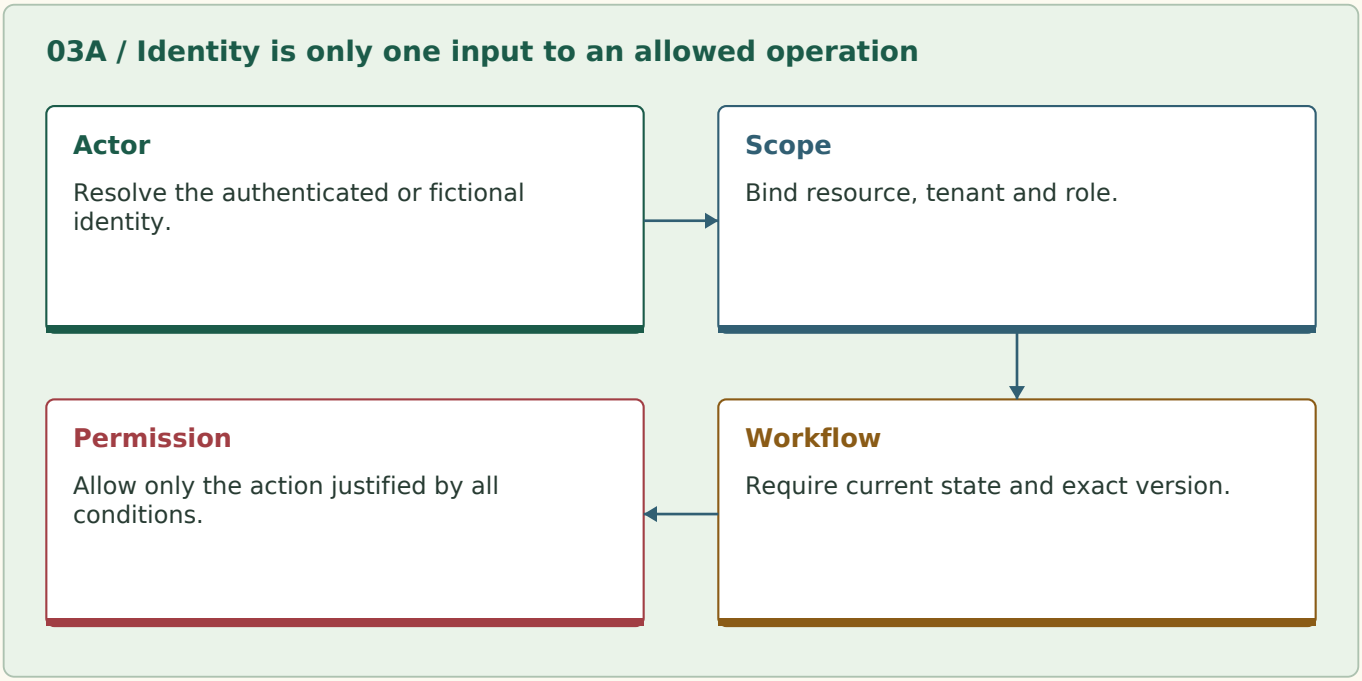
2.1 Does HTTPS stop a wrong refund calculation?	**2.2** What is an endpoint?	**2.3** Does hiding a button protect its API?

Answer explanations are in the full ebook. Sources: HTTP, FETCH, REST

03. Separate identity, permission and workflow authority

Learn to: Explain sessions, actors, tenants and the scope of an operation.

Think of it this way: A staff badge proves who entered a building; it does not authorize every room or let the holder approve their own expense.



Identity, permission and workflow state are different facts; preserve every required binding.

Who is making the request?

An **actor** is the identity or component performing an action. Authentication checks identity evidence. Authorization decides whether that actor may perform a particular action on a particular resource. Business logic adds conditions such as required state, amount, sequence and independent approval.

Mira belongs to the Blue tenant. Ben belongs to Amber. A tenant is an organizational boundary inside a shared service. Dana is a Blue reviewer. Correctly identifying Ben does not permit him to read Mira’s order or approve a Blue request.

How a session carries continuity

HTTP requests are separate messages. An application can issue a session identifier, a secret handle that lets later requests refer to server-side state. A cookie is browser-managed name/value state that can carry that handle automatically under cookie rules.

A cookie is not the account itself. The server must resolve it, check its validity and derive the actor. Some APIs use bearer tokens instead. Different credential formats do not remove the need to verify the current operation’s permission.

The optional RuleForge lab has a public actor picker. It deliberately supplies three fictional identities without implementing production login. Switching actors is a privileged testing fixture. Do not mistake that convenience for an authentication bypass.

Scope has several dimensions

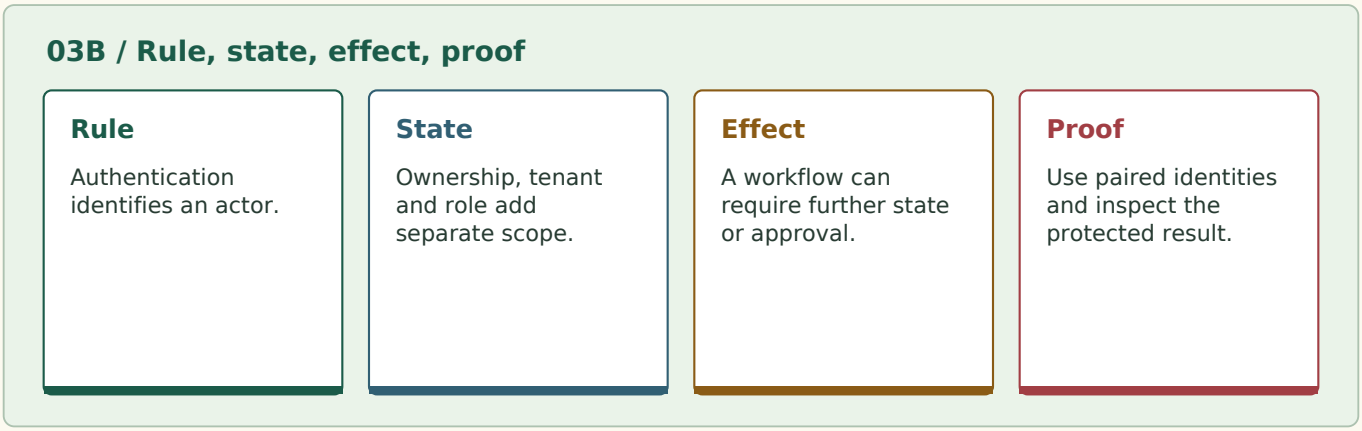
An order ID selects a record. It does not prove ownership. A role such as reviewer describes a permission category; it does not necessarily authorize every tenant. An approval for version one does not automatically approve a changed version two.

Write the operation as a sentence: Dana may approve this pending Blue request, provided Dana is an independent eligible reviewer and the approval names its exact current values. Each condition can become a separate negative test.

Common boundaries overlap

An ID-only order read may be broken object authorization. A maker approving their own request may violate separation of duties. A correct reviewer approving stale values is a binding failure. These can all affect business workflows. You do not need to force each finding into one exclusive label before understanding it.

Use two authorized fictional accounts to compare what each can do. Keep resource ownership, tenant membership, role, state and version distinct. If one changes, repeat the relevant permission decision. A session that remains valid does not freeze every permission forever.



Key takeaway. Identity, permission and workflow state are different facts; preserve every required binding.

Check your understanding

3.1 Does knowing an order ID prove ownership?	**3.2** What does the local actor picker prove?	**3.3** Can a reviewer role cross tenant boundaries automatically?

Answer explanations are in the full ebook. Sources: AUTH, AUTHZ, API1, TENANT

Sources and further reading

Research cutoff: 2 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

LOGIC - PortSwigger: Business logic vulnerabilities

Consulted targeted sections. Definitions and original educational taxonomy. The book uses its own fictional cases and explanations.

https://portswigger.net/web-security/logic-flaws

WSTG - OWASP WSTG v4.2: Business logic testing

Consulted targeted sections. Versioned testing family and workflow-oriented review questions.

https://wstg.owasp.org/stable/4-Web_Application_Security_Testing/10-Business_Logic_Testing/

BUSINESS - OWASP: Business Logic Security

Consulted targeted sections. Living defensive guidance. Rules, value authority, state and feature misuse need product-specific enforcement.

https://cheatsheetseries.owasp.org/cheatsheets/Business_Logic_Security_Cheat_Sheet.html

HTTP - RFC 9110: HTTP Semantics

Consulted targeted sections. HTTP messages, method semantics and idempotence. Transport meaning is not a complete application business contract.

https://www.rfc-editor.org/rfc/rfc9110.html

FETCH - WHATWG: Fetch

Targeted further reading. Browser request, credential and response-access boundaries.

https://fetch.spec.whatwg.org/

REST - OWASP: REST Security

Targeted further reading. API validation, operation permissions and secure response design.

https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

AUTH - OWASP: Authentication

Targeted further reading. Identity evidence and sensitive lifecycle operations, separate from business permission.

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

AUTHZ - OWASP: Authorization

Targeted further reading. Explicit permission, least privilege and checks at accepting boundaries.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

API1 - OWASP API1:2023 Broken Object Level Authorization

Targeted further reading. Object selectors do not establish caller permission or tenant ownership.

https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/

TENANT - OWASP: Multi Tenant Security

Targeted further reading. Tenant context, isolation and authorization considerations requiring actual deployment tests.

https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html